

Message queue integration

Author Taras Lysun

Репозиторій: https://github.com/taraslysun/SA-labs/tree/micro_mq

Додаткові завдання: виконати, використовуючи Kafka message queue

1. Запустити три екземпляра **logging-service** (локально їх можна запустити на різних портах), відповідно мають запуститись також три екземпляра **Hazelcast**

Запущені 3 контейнери з hazelcast і, відповідно, 3 контейнери з logging-service

```
$ docker ps --format "table {{.Names}}\t{{.Ports}}" | grep "hazelcast\|logging"
hazelcast-5701      0.0.0.0:5701->5701/tcp, :::5701->5701/tcp
hazelcast-5703      0.0.0.0:5703->5701/tcp, [::]:5703->5701/tcp
hazelcast-5702      0.0.0.0:5702->5701/tcp, [::]:5702->5701/tcp
logging-service-1   0.0.0.0:8001->8001/tcp, :::8001->8001/tcp
logging-service-2   0.0.0.0:8002->8001/tcp, [::]:8002->8001/tcp
logging-service-3   0.0.0.0:8003->8001/tcp, [::]:8003->8001/tcp
```

2. Запустити два екземпляри **messages-service** (локально їх можна запустити на різних портах)

Оскільки вирішив використовувати kafka, то запустив 2 messages-service та 3 kafka nodes

```
$ docker ps --format "table {{.Names}}\t{{.Ports}}" | grep "kafka\|messages\|zookeeper"
messages-service-2  0.0.0.0:8006->8005/tcp, [::]:8006->8005/tcp
messages-service-1  0.0.0.0:8005->8005/tcp, :::8005->8005/tcp
kafka-2             9092/tcp, 0.0.0.0:9094->9094/tcp, :::9094->9094/tcp
kafka-1             9092/tcp, 0.0.0.0:9093->9093/tcp, :::9093->9093/tcp
kafka-3             9092/tcp, 0.0.0.0:9095->9095/tcp, :::9095->9095/tcp
zookeeper-1         2888/tcp, 0.0.0.0:2181->2181/tcp, :::2181->2181/tcp, 3888/tcp
```

3. Через HTTP POST записати 10 повідомлень msg1-msg10 через **facade-service**

Запис:

```
import requests

for i in range(1,11):
    res = requests.post('http://localhost:8000/', json={'msg': f'msg{i}'})
    print(res.json())
```

Респонси:

```
$ python3 fill_system.py
{'status': 'success', 'data': {'id': '97c0fa25-a6b4-452c-9442-e05ed49e3572', 'content': 'msg1'}}
{'status': 'success', 'data': {'id': '30272951-94ed-43ed-80e3-b34f62d79299', 'content': 'msg2'}}
{'status': 'success', 'data': {'id': 'b95f06ef-e434-48e1-93d5-26971c2bd3cc', 'content': 'msg3'}}
{'status': 'success', 'data': {'id': 'c53a6603-15ee-4a9c-b33a-1a79ec5c257b', 'content': 'msg4'}}
{'status': 'success', 'data': {'id': 'b146116f-921d-409e-9432-27c6333c59c2', 'content': 'msg5'}}
{'status': 'success', 'data': {'id': '6f1bb7b4-9bc1-4ab9-a7f0-9bae82e01193', 'content': 'msg6'}}
{'status': 'success', 'data': {'id': '3a4283e5-8c1d-4bfb-9998-ed1e3a6fad64', 'content': 'msg7'}}
{'status': 'success', 'data': {'id': '25604023-3677-4f9f-958a-b40bd8e6289e', 'content': 'msg8'}}
{'status': 'success', 'data': {'id': 'e5cab8dd-202a-4388-abd8-ddefe57154ce', 'content': 'msg9'}}
{'status': 'success', 'data': {'id': '85a4353b-286a-4c5c-830a-aab436a6374f', 'content': 'msg10'}}
```

4. Показати які повідомлення отримав кожен з екземплярів **logging-service**
(це має бути видно у логах сервісу)

Logging-service-1:

```
$ docker logs logging-service-1
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
```

Logging-service-2:

```
$ docker logs logging-service-2
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
Starting Hazelcast container on port 192.168.1.55:5702
Hazelcast container hazelcast-5702 is running on port 5702
Saved message: 97c0fa25-a6b4-452c-9442-e05ed49e3572 - msg1
INFO:      172.18.0.4:59882 - "POST /logs HTTP/1.1" 201 Created
Saved message: 3a4283e5-8c1d-4bfb-9998-ed1e3a6fad64 - msg7
INFO:      172.18.0.4:59896 - "POST /logs HTTP/1.1" 201 Created
Saved message: 25604023-3677-4f9f-958a-b40bd8e6289e - msg8
INFO:      172.18.0.4:59902 - "POST /logs HTTP/1.1" 201 Created
```

logging-service-3:

```
$ docker logs logging-service-3
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8001 (Press CTRL+C to quit)
Starting Hazelcast container on port 192.168.1.55:5703
Hazelcast container hazelcast-5703 is running on port 5703
Saved message: 30272951-94ed-43ed-80e3-b34f62d79299 - msg2
INFO:      172.18.0.4:50946 - "POST /logs HTTP/1.1" 201 Created
Saved message: b95f06ef-e434-48e1-93d5-26971c2bd3cc - msg3
INFO:      172.18.0.4:50958 - "POST /logs HTTP/1.1" 201 Created
Saved message: c53a6603-15ee-4a9c-b33a-1a79ec5c257b - msg4
INFO:      172.18.0.4:50962 - "POST /logs HTTP/1.1" 201 Created
Saved message: b146116f-921d-409e-9432-27c6333c59c2 - msg5
INFO:      172.18.0.4:50970 - "POST /logs HTTP/1.1" 201 Created
Saved message: 6f1bb7b4-9bc1-4ab9-a7f0-9bae82e01193 - msg6
INFO:      172.18.0.4:50978 - "POST /logs HTTP/1.1" 201 Created
Saved message: e5cab8dd-202a-4388-abd8-ddefe57154ce - msg9
INFO:      172.18.0.4:50980 - "POST /logs HTTP/1.1" 201 Created
Saved message: 85a4353b-286a-4c5c-830a-aab436a6374f - msg10
INFO:      172.18.0.4:50996 - "POST /logs HTTP/1.1" 201 Created
```

5. Показати які повідомлення отримав кожен з екземплярів

messages-service (це має бути видно у логах сервісу)

messages-service-1:

```
$ docker logs messages-service-1
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8005 (Press CTRL+C to quit)
Received message: msg2
Received message: msg3
Received message: msg5
Received message: msg6
Received message: msg7
Received message: msg10
INFO:      172.18.0.4:59620 - "GET /messages HTTP/1.1" 200 OK
```

messages-service-2:

```
$ docker logs messages-service-2
INFO:      Started server process [1]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
INFO:      Uvicorn running on http://0.0.0.0:8005 (Press CTRL+C to quit)
Received message: msg1
Received message: msg4
Received message: msg8
Received message: msg9
INFO:      172.18.0.4:39064 - "GET /messages HTTP/1.1" 200 OK
```

Також, для зручності, додав додаткове поле в facade-service відповіді для моніторингу розподілу повідомлень:

```
{
  "status": "success",
  "data": {
    "msg_clusters": [
      4,
      6
    ],
    "messages": [
      [
        "msg1",
        "msg4",
        "msg8",
        "msg9"
      ],
      [
        "msg2",
        "msg3",
        "msg5",
        "msg6",
        "msg7",
        "msg10"
      ]
    ],
    "logs": "msg10,msg7,msg2,msg3,msg4,msg8,msg6,msg9,msg1,msg5"
  }
}
```

6. Декілька разів викликати HTTP GET на **facade-service** та отримати об'єднані дві множини повідомлень - це мають бути повідомлення з

logging-service та messages-service

```
{
  "status": "success",
  "data": {
    "msg_clusters": [
      4,
      6
    ],
    "messages": [
      [
        "msg1",
        "msg4",
        "msg8",
        "msg9"
      ],
      [
        "msg2",
        "msg3",
        "msg5",
        "msg6",
        "msg7",
        "msg10"
      ]
    ],
    "logs": "msg10,msg7,msg2,msg3,msg4,msg8,msg6,msg9,msg1,msg5"
  }
}
```

Перевірка на відмовостійкість:

1. не запускайте (вимкніть) два екземпляри **messages-service** для того щоб повідомлення тимчасово не вичитувались та зберігались у message queue

```
$ docker ps --format "table {{.Names}}\t{{.Ports}}" | grep "kafka\|messages\|zookeeper"
kafka-3          9092/tcp, 0.0.0.0:9095->9095/tcp, :::9095->9095/tcp
kafka-2          9092/tcp, 0.0.0.0:9094->9094/tcp, :::9094->9094/tcp
kafka-1          9092/tcp, 0.0.0.0:9093->9093/tcp, :::9093->9093/tcp
zookeeper-1     2888/tcp, 0.0.0.0:2181->2181/tcp, :::2181->2181/tcp, 3888/tcp
```

2. відправте 10 (можна і 100) повідомлень через **facade-service**

```
{ 'status': 'success', 'data': { 'id': '0a0b4a2a-7b10-4193-a0b4-0a9a30a73911', 'content': 'msg93' }}
{ 'status': 'success', 'data': { 'id': '89ac569d-1d38-42fd-954d-a191f2aee5d4', 'content': 'msg94' }}
{ 'status': 'success', 'data': { 'id': '85c94512-8889-4a40-9c2c-0b7954078a33', 'content': 'msg95' }}
{ 'status': 'success', 'data': { 'id': '9fc3faee-66f7-4023-8d20-13376341a147', 'content': 'msg96' }}
{ 'status': 'success', 'data': { 'id': 'f9b15775-4ba3-405b-94ee-16c7e7109147', 'content': 'msg97' }}
{ 'status': 'success', 'data': { 'id': '2897571a-7cbe-4108-8193-c014ebc051b3', 'content': 'msg98' }}
{ 'status': 'success', 'data': { 'id': '8428c21d-d3d2-4e3a-af12-d2518c0deb2a', 'content': 'msg99' }}
{ 'status': 'success', 'data': { 'id': '0d567fa6-047d-431f-b2c7-9726edf3deb2', 'content': 'msg100' }}
```

3. вимкніть один з серверів (message queue)

- a. для Hazelcast це має бути сервер з меншою IP-адресою
- b. для Kafka це має бути т.з. Leader

Можна побачити, що кожен топік має свою частину:

```
$ docker exec kafka-1 kafka-topics --bootstrap-server kafka-1:9092 --describe --topic messages
Topic: messages TopicId: sjdGmc4aRtSqGqy-jJ0eVA PartitionCount: 3 ReplicationFactor: 3 Configs:
Topic: messages Partition: 0 Leader: 3 Replicas: 3,2,1 Isr: 3,2,1 Elr: N/A LastKnownElr: N/A
Topic: messages Partition: 1 Leader: 1 Replicas: 1,3,2 Isr: 1,3,2 Elr: N/A LastKnownElr: N/A
Topic: messages Partition: 2 Leader: 2 Replicas: 2,1,3 Isr: 2,1,3 Elr: N/A LastKnownElr: N/A
```

Якщо вимкнути, наприклад, 3-й брокер, то буде наступне

```
$ docker exec kafka-1 kafka-topics --bootstrap-server kafka-1:9092 --describe --topic messages
Topic: messages TopicId: sjdGmc4aRtSqGqy-jJ0eVA PartitionCount: 3 ReplicationFactor: 3 Configs:
Topic: messages Partition: 0 Leader: 2 Replicas: 3,2,1 Isr: 2,1 Elr: N/A LastKnownElr: N/A
Topic: messages Partition: 1 Leader: 1 Replicas: 1,3,2 Isr: 1,2 Elr: N/A LastKnownElr: N/A
Topic: messages Partition: 2 Leader: 2 Replicas: 2,1,3 Isr: 2,1 Elr: N/A LastKnownElr: N/A
```

- 4. запуснуть два екземпляри **messages-service** і перевірте що вони отримали всі повідомлення з черги

Навіть після запису 100 повідомлень в 3 брокери, потім вимкнення одного брокера та увімкнення 2-х message-service, вони змогли прочитати в сумі 100 повідомлень завдяки replication factor=3, встановленому при створенні топіка.

```
{
  "status": "success",
  "data": {
    "msg_clusters": [
      23,
      77
    ],
    "messages": [
      [
        "msg2",
        "msg4"
      ]
    ]
  }
}
```